8. Packages, Modules, Visibility, and go.mod

08. Packages, Modules, Visibility, and go.mod

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `08-packages-modules`

Learning objectives

- Create modules with go.mod
- Export identifiers with Capital letters
- Import packages and avoid import cycles
- Organize code into packages with a single purpose

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Modules and visibility scale Go across teams. Capitals export; `internal/` enforces boundaries without heavy ceremony.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `go mod init` sets the module path; packages map to directories.
- Exported identifiers start with A–Z; lowercase stays package-private.
- External tests (`package foo_test`) only see exported API — great design pressure.
- `go mod tidy` adds required modules and drops unused ones.
- `internal/` directories are importable only by their parent tree.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-packages, && cd /tmp/gw-packages,
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    // imagine: "example.com/hello/mathx"
)

// In package mathx:
// func Square(n int) int { return squareHint(n) }
// func squareHint(n int) int { return n * n }

func main() {
    // External callers use Square only.
    fmt.Println(4 * 4)
}
```

Run it:

```
go run .
```

Expected output:

```
16
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Create package mathx with Square + unexported helper.
2. Write an external test in package mathx_test.
3. Try calling the unexported helper from outside — read the compiler error.
4. Run `go mod tidy` after adding/removing a dependency import.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Modules

A **module** is a versioned collection of packages (`go.mod`).

```
go mod init github.com/you/project
go get example.com/some/dep@v1.2.3
go mod tidy
```

Packages

Every Go file starts with `package name`. Directory ≈ package (usually). `package main` with `func main()` builds an executable.

Visibility

- `Name` (capital) → exported, visible outside the package

- `name` (lowercase) → package-private

There is no `private` keyword. This simplicity scales surprisingly well.

Import paths

```
import (
    "fmt"
    "github.com/you/project/internal/auth"
)
```

`internal/` directories are importable only by the parent tree — enforce encapsulation.

Failure modes to watch while coding

- Creating many tiny packages too early (start coarser)
- Import cycles from poor boundaries
- Putting everything in `package main` forever

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python `_name` is convention; Go unexported is enforced by the compiler.
- Import paths are module paths, not relative `../` hacks for libraries.
- Avoid eternal package main — split libraries when reuse appears.
- Circular imports fail loudly; redesign packages instead of tricks.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would “work”. The goal is a Go-shaped intuition.

Going deeper

Keep packages purposeful: one idea per package. `util` packages become junk drawers — name by domain.

Practice checklist

Before the exercise, confirm you can:

1. Explain “Packages, Modules, Visibility, and go.mod” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Creating many tiny packages too early (start coarser)
- Import cycles from poor boundaries
- Putting everything in `package main` forever

Exercises

Work in `exercises/08-packages-modules/`. Read that folder's `README.md` before you code.

What you will build

Implement unexported squareHint used by exported Square; tests import from outside.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/08-packages-modules/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. How do you export a function?
2. What does `go mod tidy` do?
3. Why does `internal/` exist?

Further reading

- https://go.dev/doc/modules/gomod-ref
- https://go.dev/doc/code